ASSIGNMENT 5 – CO5

PART A | TASK A5

DESIGN FOR CORRECTNESS AND SAFETY

JDBC Transaction Design and SQL Injection Prevention

Course: Programming in Java
Topic: Data Persistence Engineering – Files and Databases

Part A design report containing transaction pseudocode, failure-safety reasoning, SQL injection analysis, a PreparedStatement redesign and an alternative auto-commit comparison.

1. Problem Understanding

Task A5 has two parts. A5.1 asks for a JDBC transaction that transfers ■500 from a hostel wallet to a canteen wallet without losing or creating money if failure occurs between the updates. A5.2 presents a login query built by concatenating user input into SQL and asks how crafted input can bypass the password check and how to redesign it with PreparedStatement.

2. A5.1 – Transaction Design (2 Marks)

Design principle

Treat the debit and credit as one atomic transaction. Disable auto-commit before either update. Execute both updates on the same Connection. Verify that each update affects exactly one wallet row. Commit only after both succeed. If an update fails, affects an unexpected number of rows, or an exception occurs, roll back the transaction. This ensures the database does not retain only one side of the transfer.

SIMATS-style algorithmic pseudocode

```
ALGORITHM TRANSFER_WALLET_AMOUNT(connection, hostelWalletId, canteenWalletId)
    transferAmount <- 500
    oldAutoCommit <- GET_AUTO_COMMIT(connection)

    TRY
        SET_AUTO_COMMIT(connection, FALSE)

        debitRows <- EXECUTE_UPDATE(
            "UPDATE wallet SET balance = balance - ? " +
            "WHERE wallet_id = ? AND balance >= ?",
            transferAmount, hostelWalletId, transferAmount)

        IF debitRows NOT EQUAL TO 1 THEN
            RAISE transfer failure
        ENDIF

        creditRows <- EXECUTE_UPDATE(
            "UPDATE wallet SET balance = balance + ? WHERE wallet_id = ?",
            transferAmount, canteenWalletId)

        IF creditRows NOT EQUAL TO 1 THEN
            RAISE transfer failure
        ENDIF

        COMMIT(connection)
        DISPLAY "Transfer successful"

    CATCH database error OR transfer failure
        ROLLBACK(connection)
        DISPLAY "Transfer failed; no wallet update was retained"

    FINALLY
        SET_AUTO_COMMIT(connection, oldAutoCommit)
    ENDTRY
END ALGORITHM
```

Explanation

The debit includes a balance condition so the hostel wallet cannot become negative. The affected-row checks detect a missing wallet or a debit/credit that did not update exactly one row. Both updates use the same connection and transaction. If the debit succeeds but the credit fails, rollback cancels the debit. If both succeed, commit makes both permanent. The original auto-commit setting is restored in the FINALLY block.

Failure cases

Failure point	Database outcome
Before either update	No transfer changes have been committed.
After debit, before credit	Rollback cancels the debit; neither wallet retains a partial transfer.

Failure point	Database outcome
Credit update fails	Rollback cancels the earlier debit.
Both updates succeed	Commit permanently records the debit and credit together.

3. A5.2 – Injection Analysis (2 Marks)

The unsafe statement is:

```
String sql = "SELECT * FROM users WHERE id='" + id
          + "' AND pwd='" + pwd + "'";
```

How crafted input bypasses the check

The application inserts id and pwd directly into the SQL text. An attacker can supply an ID such as **admin' OR '1'='1' --** and any incorrect password. In a database that recognises -- followed by whitespace as a line comment, the resulting condition is effectively id = 'admin' OR '1'='1'. The always-true expression can make the query return a user row without validating the supplied password. Exact comment syntax varies by database, but the underlying defect is that input is interpreted as SQL syntax.

PreparedStatement redesign

```
String sql = "SELECT * FROM users WHERE id = ? AND pwd = ?";
try (PreparedStatement ps = connection.prepareStatement(sql)) {
    ps.setString(1, id);
    ps.setString(2, pwd);
    try (ResultSet rs = ps.executeQuery()) {
        boolean authenticated = rs.next();
    }
}
```

The SQL structure is fixed and the values are bound separately through placeholders. The supplied text is treated as parameter data, not executable SQL. The injected OR/comment characters therefore do not change the WHERE clause; the invalid credentials are rejected unless they genuinely match a stored record. This example follows the assignment's requested PreparedStatement redesign; a production login system should store and verify password hashes rather than compare plaintext passwords.

4. Alternative Concept and Complexity Analysis

Two separate auto-commit updates

If the debit and credit are executed as separate statements while auto-commit is enabled, the debit may be committed immediately. If the program then fails before the credit, the hostel wallet remains debited while the canteen wallet is not credited. The transfer becomes incorrect and money appears to be lost. A later retry can also risk a duplicate or inconsistent transfer unless additional safeguards are implemented.

Aspect	One transaction	Two independent auto-commit updates
Database work	Two indexed wallet updates plus transaction begin/commit or rollback work.	Two indexed wallet updates, each committed independently.
Time complexity	For fixed two wallet rows, O(1) database-row work with transaction/logging overhead; actual latency depends on locks, logging and the DBMS.	Also O(1) for fixed two indexed rows; incurs separate commit overhead and may be faster or slower depending on DBMS.
Space complexity	O(1) application-side extra state for this transfer; database transaction/log buffers are implementation-dependent.	O(1) application-side extra state; each statement commits separately.
Correctness on failure	Atomic: both updates commit or neither does.	Partial transfer is possible if failure occurs between statements.

Interpretation

Both approaches perform two database updates, so the asymptotic row-work for this fixed-size transfer is O(1) when wallet IDs are indexed. A transaction adds coordination, logging and commit/rollback overhead; exact elapsed time must be measured on the chosen database. The small potential overhead is justified because atomicity protects the correctness of the wallet balances. Big-O alone does not express transaction latency.

5. Conclusion

The ■500 transfer must execute as one JDBC transaction: disable auto-commit, perform and verify both wallet updates, commit only after both succeed, and roll back on failure. The unsafe login query is vulnerable because user input is concatenated into SQL. PreparedStatement binds values separately and prevents those values from changing the query structure. Separate auto-commit updates can leave a partial transfer, while a transaction preserves the all-or-nothing result.

6. Submission Note

Task A5 is in Part A – Persistence Design & Pseudocode. The required primary answer is algorithmic pseudocode and reasoning. The Java PreparedStatement snippet is supplementary to demonstrate the requested safe query form; it does not replace pseudocode. No complete runnable Java program or execution screenshot is requested specifically for Part A Task A5.